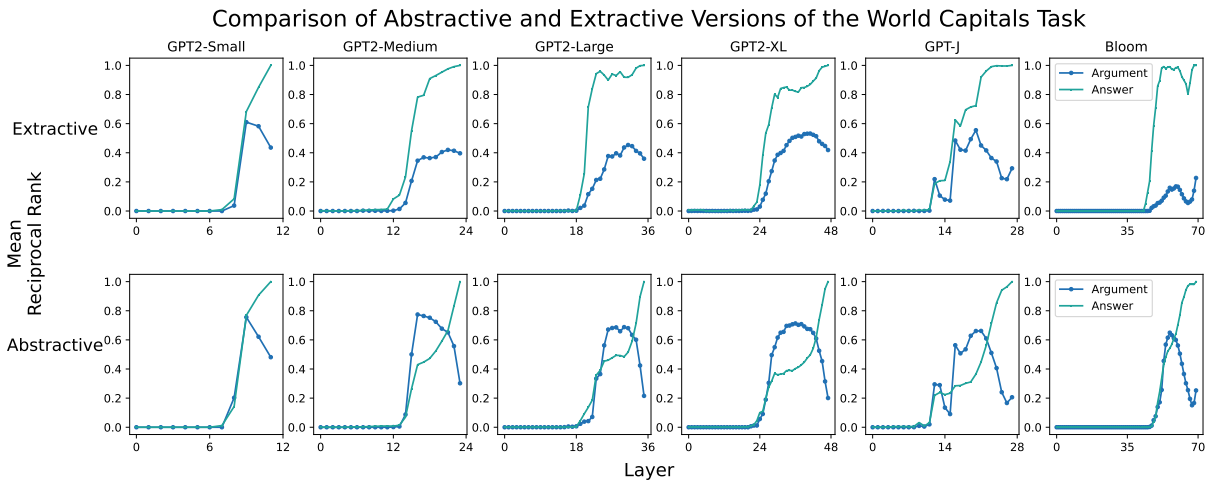

Figure 9: The 'X' pattern of argument and answer tokens crossing in the course of the forward pass is the characteristic pattern in argument-function processing. In the main text, we show how the models we test use this type of processing to recall the capital cities of locations. When we make the task extractive (by including the correct capital in the given context), the model does not have to setup an argument and function in order to get the answer, and the pattern disappears. This highlights the differences we describe in processing extractive and abstractive tasks. Both datasets are filtered for examples where the models were correct.

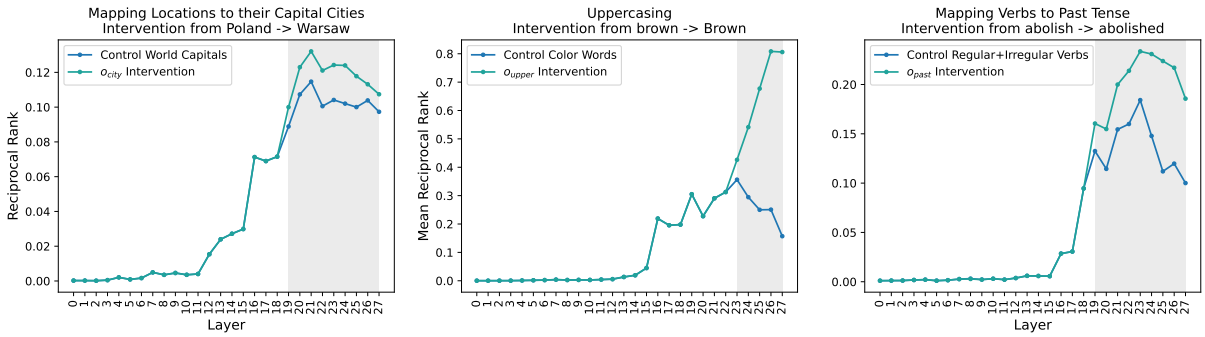

Figure 10: We use the same stimuli to extract $\vec{o}$ vectors on GPT-J. Results are similar for the uppercasing function, but only very weakly positive on the world capitals task.

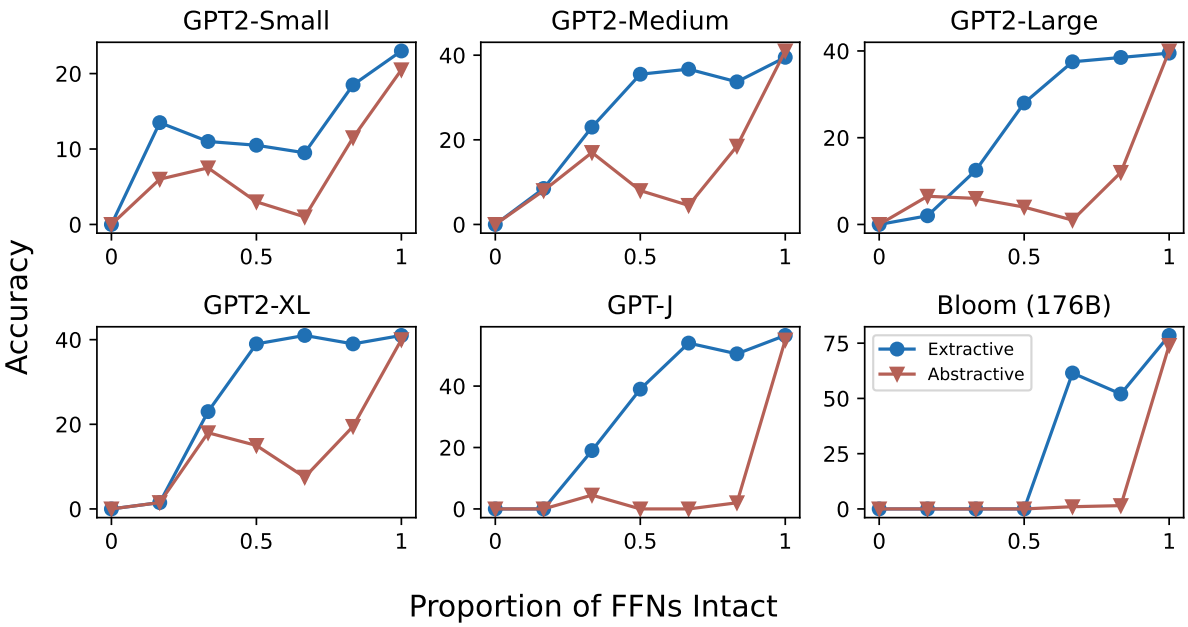

Figure 11: Results of removing FFN sublayers for the colored objects task for all models.

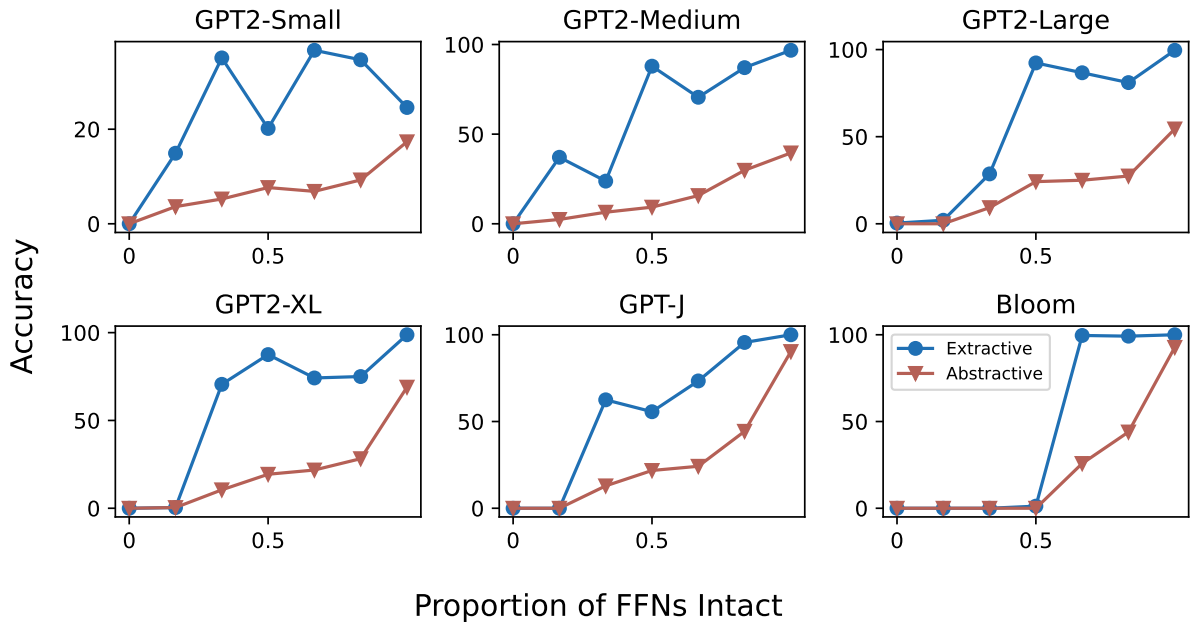

Figure 12: Results of removing FFN sublayers for the world capitals task for all models.

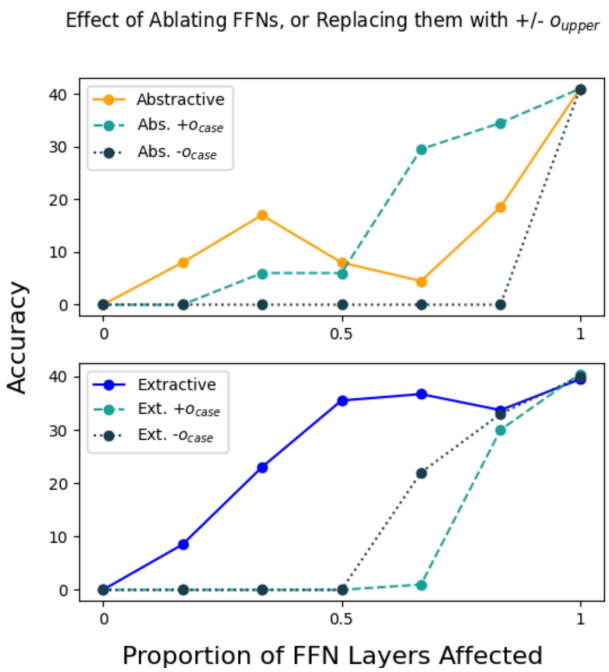

Figure 13: Replacing FFN updates with $+o_{case}$ helps recover accuracy in abstractive tasks where the answer is expected to be uppercase compared to subtracting it or ablating the FFNs. In extractive tasks, the task is primarily solved by attention modules and adding or subtracting o_{case} only hurts performance.

transforms this representation into the final output. When subject enrichment (Geva et al., 2023) is not possible, these same pseudo mover heads would then copy the unenriched subjects (i.e., the regular argument tokens). In these cases, the model would have to apply the function after already copying it over, creating the argument-processing signature. Future work is needed to see if it is possible to unify these different interpretations and perspectives.

D Effect on Zero-shot Performance

We find that intervening on the model with $\vec{o}$ vectors has applications in controllable generation, that is, guiding the generation process towards some relevant text. We showed this was the case in Section 4, but we can also apply this idea to the context of zero-shot learning. When we provide in-context examples, we are also providing the output format of the prompt. Consider the example "Q: What is the capital of Poland? A:". unlike the one shot example given in Figure 2, there is no indication that the next word should be " Warsaw" over continuing the generation as a complete sentence "The capital of Poland is Warsaw", which is what GPT2-Medium actually generates. If we decode at every layer, as is shown in Table 1 we can see that the model still goes through argument formation despite preferring to generate the full sentence. We can take advantage of this behavior by replacing the FFN layers in the later layers with $\vec{o_{city}}$ in order

Layer	Top Token
0	(
1	A
2	A
3	A
4	A
5	A
6	A
7	A
8	A
9	The
10	The
11	The
12	The
13	The
14	The
15	The
16	The
17	The
18	Poland
19	Poland
20	Poland
21	Poland
22	Poland
23	The

Table 1: These are the top tokens per layer in GPT2-Medium on the example zero-shot Poland example.

to guide the generation to the expected response of immediately generating the capital. We can perform this experiment on the past tensing task as well. Results on the zero-shot tasks are shown in Figure 14. We find that on the world capitals task, we can greatly improve the propensity of the model to output the expected answer by performing an $\vec{o}$ vector intervention, improving zero-shot performance from 5.6% to 33.0%. On the past tense mapping task, where perhaps the output format is more obvious from the prompt, the zero and one shot performances are about equal, but we still see a modest improvement over the one shot results of about 4.2%. Although the tasks are very simple, we achieve this by effectively ablating FFN layers (layers 19-23) and precomputing their activations, suggesting it might be possible to edit models extensively to limit their expressiveness to one type of output while also making them more efficient. We are optimistic about future work in this area.

E Effect of Layer Choice on Intervention Results

In the main text, we replace FFNs starting at either layer 18 or 19 GPT2-Medium to the end (indexed at 0). We find that intervening on only one layer promotes the output token, but not to the top of the distribution. One possibility is that the model makes gradual updates that are pushing the token representation in generally the same direction (Jastrzebski et al., 2017). In Figure 15, we show that adding any of the $\vec{o}$ vector interventions at any single layer at 18 or afterwards, there is a roughly equivalent increase to the average reciprocal rank of the target word. The logit difference between the argument and answer token (in the logits of each early-decoded layer) shows this as well as a gradual increase. This is exemplified in Figure 2 in the main paper.

F Effect of Tokenization on the Effectiveness of $\vec{o}$ Vectors

The tokenizer can split one word into multiple subtokens, such as “Purple" into the tokens “Pur" and “ple". This occurs with words that were less frequent in the training data. We find that this process has a generally negative effect on the performance of the intervention we perform. Intuitively, if we are trying to use $\vec{o_{upper}}$ to capitalize the “purple" token into “Purple", it must map from “purple" (one token) to “Pur". It seems less obvious, then,